



CDLI

GSOC 2026

# MCP SERVER AND AGENT

---

ARMAAN GUPTA

Mentors: Jayanth Kumar & Émilie Pagé-Perron

Google Summer of Code 2026

# Index

- 1. Curriculum Vitae of the Student**
- 2. Project Overview**
  - 2.1 Few Terminologies
  - 2.2 Tech Stack used for the project
- 3. Project Goals & Deliverables**
- 4. Project Architecture**
  - 4.1 MCP Tools to be implemented
- 5. Phases of Implementation**
  - 5.1 Core MCP Server & Modular Tool Design**
    - 5.1.1 advanced\_search
    - 5.1.2 ground\_term
    - 5.1.3 get\_metadata
    - 5.1.4 search\_entity
    - 5.1.5 Others
  - 5.2 Developing the AI Research Workspace**
    - 5.2.1 Proposed Frontend
    - 5.2.2 API Key storage & encryption
    - 5.2.3 Context management during an ongoing session
    - 5.2.4 General token usage breakdown
  - 5.3 The Research Paper Generation Agent**
    - 5.3.1 Graph State Definition
    - 5.3.2 Agentic Nodes
  - 5.4 Deployment, Testing & Documentation**
    - 5.4.1 Anticipated technical challenges & mitigation
    - 5.4.2 Testing & Validation
    - 5.4.3 Deploymet
- 6. Project Timeline**
- 7. Planned GSoC Work Hours**
- 8. More About Me**
- 9. Commitments & Availability**
- 10. AI being the helping hand**
- 11. References**

## 1 CURRICULUM VITAE OF THE STUDENT

|          |                                                                    |
|----------|--------------------------------------------------------------------|
| Name     | Armaan Gupta                                                       |
| Email    | <a href="mailto:armaanngupta@gmail.com">armaanngupta@gmail.com</a> |
| GitHub   | <a href="https://github.com/armaanngupta">armaanngupta</a>         |
| Gitlab   | <a href="https://gitlab.com/armaanngupta">armaanngupta</a>         |
| LinkedIn | <a href="https://linkedin.com/in/armaan">linkedin/armaan</a>       |
| Timezone | Indian Standard Time (UTC +5:30)                                   |
| Location | Madhya Pradesh, India                                              |

## 2 PROJECT OVERVIEW

This project builds an **AI-powered research interface** for CDLI using the **Model Context Protocol (MCP)**. The goal is to make it easier to explore the CDLI database without requiring users to learn complex search forms or syntax.

Right now, finding specific information across more than 400,000 cuneiform tablets usually requires structured queries and familiarity with the CDLI interface. This creates a **barrier for many researchers and students** who may know what they are looking for, but not how to express it in the required format.

To address this, the project introduces an MCP server that converts natural language queries into structured CDLI API calls. Instead of manually selecting filters, users can ask questions in plain English, and the system handles the translation into the correct parameters and endpoints.

### 2.1 Few Terminologies –

**MCP : Model Context Protocol (MCP)** is a standardized way for AI models to interact with external systems. It allows models to access tools, prompts, and structured data without requiring custom integrations for each use case.

**MCP SDK:** SDKs for different programming languages that implement MCP.

**MCP Tools:** Functions that your LLM can actively call, and decides when to use them based on user requests. Tools can write to databases, call external APIs, modify files, or trigger other logic.

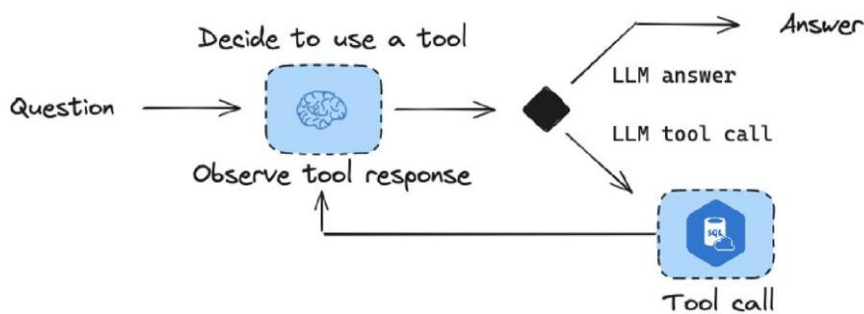


Fig. LLM Interaction with MCP

\* MCP uses **JSON-RPC 2.0** as its underlying RPC protocol. Client and servers send requests to each other and respond accordingly.

```

{
  "jsonrpc": "2.0",
  "id": 3,
  "method": "tools/call",
  "params": {
    "name": "get_artifact",
    "arguments": {
      "id": "647421",
    }
  }
}
  
```

Fig. Sample JSON-RPC tool call request

MCP supports two transport mechanisms:

- **STDIO transport:** Used for local integrations where the MCP server runs as a process on the user's machine. This is suitable for desktop AI tools and has minimal overhead.
- **Streamable HTTP transport:** Used for remote communication between the browser-based widget and the hosted MCP server. Server-Sent Events (SSE) allow streaming responses, which improves responsiveness for longer queries.

## 2.2 Tech Stack used for the Project –

**TypeScript**, Node.js (Express), SSE (Server-Sent Events) for streaming, **React/Vite** for the embedded widget. Users provide their own LLM keys (**BYOM**) so CDLI incurs zero LLM API costs. **LangGraph** and Python for paper generation workflow.

### 3 PROJECT GOALS AND DELIVERABLES

- Allow users to ask natural language questions instead of learning complex search forms and syntax.
- Provide a structured workflow for generating detailed research outputs (such as papers)
- **Make CDLI the first AI-native digital humanities archive.**

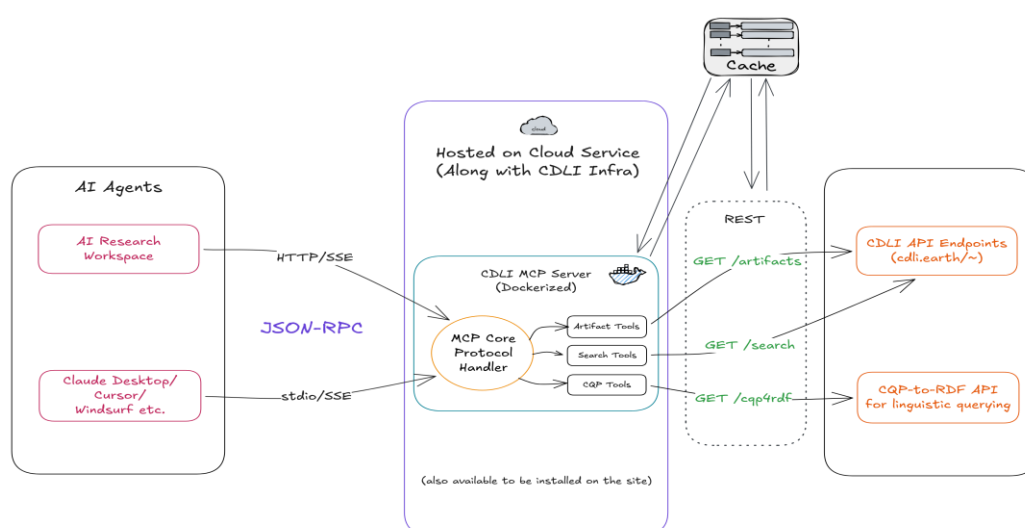
#### Deliverables :

- **MCP Server** : A fully functional backend hosted on CDLI's infrastructure that turns database queries into AI-ready tools.
- **AI Research Workspace** : A chat widget embedded directly on the CDLI website that harnesses the MCP server to answer researcher queries.
- **Documentation** : Thorough development guides and setup instructions so future maintainers can easily understand and expand the project.

### 4 PROJECT ARCHITECTURE

At a high level, the project has **two** major components: the **MCP Server** and the **AI Research Widget** connected by a standard HTTP/SSE transport layer.

The MCP Server sits between the AI client and the CDLI database APIs. It defines a fixed set of structured tools. When a researcher asks a question, the AI picks the right tool, the server calls the CDLI API, and the result flows back as clean, formatted text.



High-Level System Architecture

The two key interaction patterns are:

- **Web Widget path:** The floating chat widget on [cdli.earth](https://cdli.earth) connects to the hosted MCP server at [mcp.cdli.earth](https://mcp.cdli.earth) over SSE. The widget sends the user's message to the LLM, which in turn calls MCP tools and streams the response back to the browser.
- **Desktop client path:** A researcher using Claude Desktop or Cursor adds the CDLI server as an MCP source in their local config. Their local AI client connects via Stdio or SSE and gets the same tools.

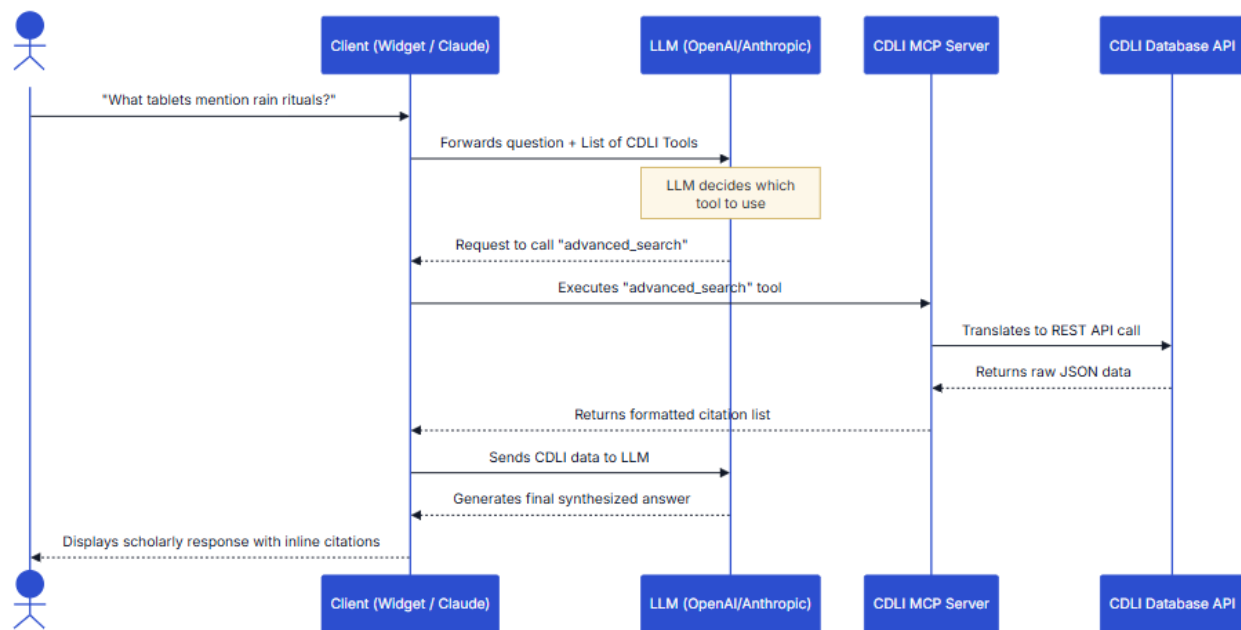


Fig. Sequence Diagram (Request Flow Diagram)

\*(A more detailed sequence diagram is available [here](#).)

## 4.1 MCP Tools to be implemented –

| Tool                   | Purpose                                                          |
|------------------------|------------------------------------------------------------------|
| <b>advanced_search</b> | Filtered artifact search by multiple fields. Core tool           |
| <b>ground_term</b>     | Normalize a misspelled user term to its CDLI canonical value     |
| <b>get_metadata</b>    | Fetch metadata for entities like artifacts, authors, genres, etc |
| <b>get_linked_data</b> | Fetch Linked Open Data for CDLI entities                         |
| <b>get_inscription</b> | Fetch transliteration/text of an inscription                     |
| <b>search_entity</b>   | Search about entities by related specific fields other than id   |
| <b>cqp_query</b>       | Linguistic pattern search using CQP expressions                  |
| <b>ping</b>            | Check if the server is alive                                     |

The diagram below shows the **flow of a complex query** through the system from user input to the final response. The LLM iterates through tools (searching, then fetching translations) until it has everything it needs to give a complete answer.

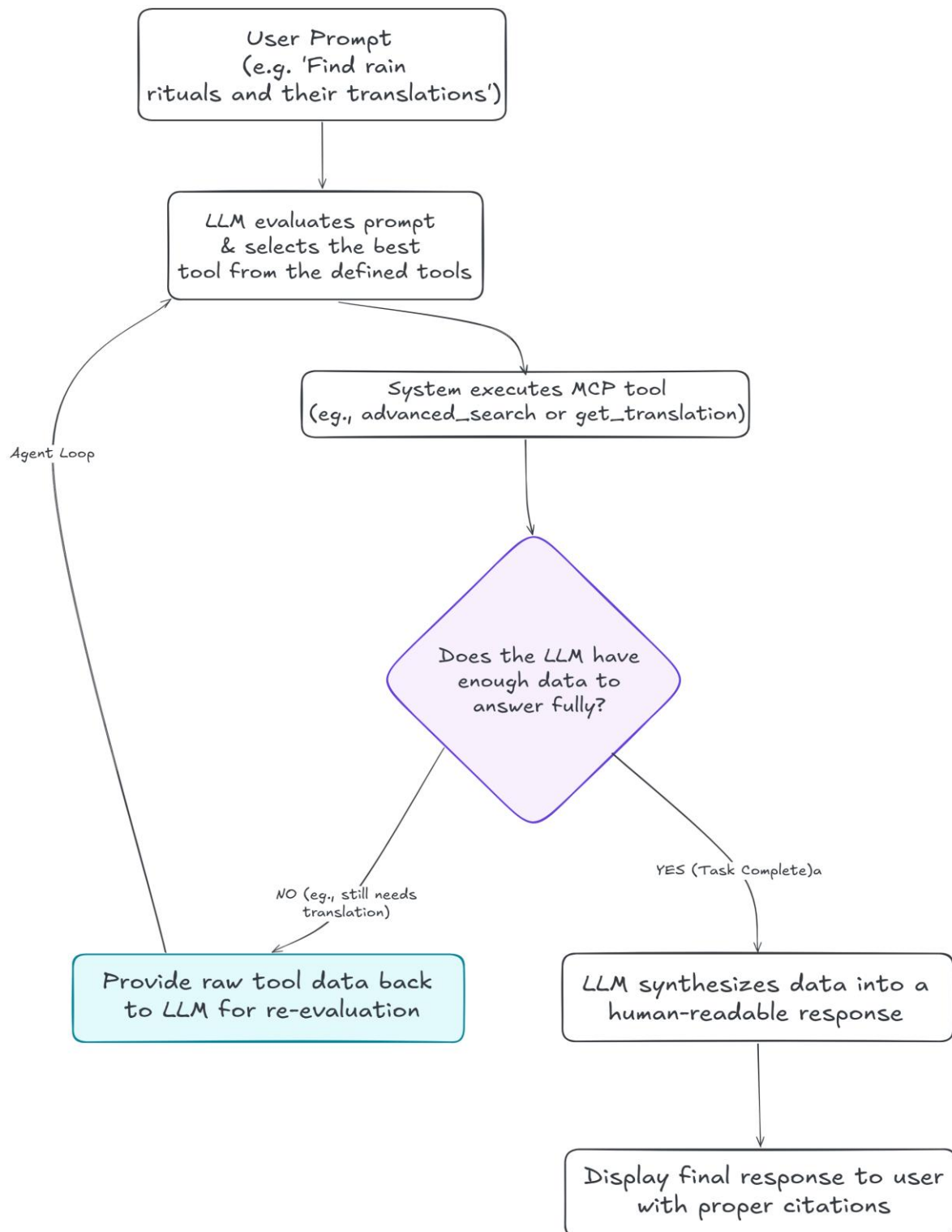


Fig. Prompt Flow Diagram

## 5 PHASES OF IMPLEMENTATION

### 5.1 Core MCP Server & Modular Tool Design

It involves building the server that bridges natural language questions and the CDLI database. Each CDLI capability (searching, fetching artifact details, running a linguistic query) will be defined as a separate, self-contained MCP tool.

The main focus of the tool designing is to **keep the number of tools as low as possible**, and enough to cover most of the functionalities of the CDLI database and API endpoints. The tools will be built using the **Official MCP Typescript SDK**, in a format as described below :

```
{
  name: "searchFlights",
  description: "Search for available flights",
  inputSchema: {
    type: "object",
    properties: {
      origin: { type: "string", description: "Departure city" },
      destination: { type: "string", description: "Arrival city" },
      date: { type: "string", format: "date", description: "Travel date" }
    },
    required: ["origin", "destination", "date"]
  }
}
```

#### 5.1.1 advanced\_search

This is the core tool that is exposed in the MCP server for searching the artifacts, based on all the fields, queries and filters defined [here](#). It serves as **“the entry point for discovery”** for the LLM.

**Pagination** of the response is handled through the offset-based approach, by accepting **limit** and **offset** inputs, and using them in the request url, and if it returns a subset, a note is added, for example “Showing 20 of 342 total results”.



```

export const name = "advanced_search";
export const description =
  "Search CDLI artifacts using specific fields. All parameters are optional  

  – " +
  "combine any subset to filter results. Supports wildcards (* ?), boolean  

  operators " +
  "(%AND% / %OR% within a field), quoted literals (\"exact phrase\"), and  

  regex (/pattern/).";
export const inputSchema = {
  type: "object",
  properties: {
    designation: {
      type: "string",
      description: "Primary publication designation (e.g. 'MVN 3, 1').  

      Supports wildcards/regex.",
    },
    museum_no: {
      type: "string",
      description: "Museum number (e.g. 'BM 00001'). Tolerant of leading  

      zero differences.",
    }
  }
}

```

#### Tool Definition

\* Here only the fields of designation and museum\_no are shown, but the actual implementation will include all the available fields (including limit and offset) as query params.

The fields are mapped 1:1 for advanced search URL param as –

```

const FIELD_PARAMS: Record<string, string> = {
  designation: "designation",
  museum_no: "museum_no",
}

```

The JSON response for the searched fields is fetched as below –

```
const url = `https://cdli.earth/search.json?${params.toString()}`;
const response = await fetch(url, {
  headers: {
    "Accept": "application/json"
  },
});
if (!response.ok) {
  return {
    content: [{
      type: "text",
      text: `CDLI API error: HTTP ${response.status} –
${response.statusText}`,
    }],
    isError: true,
  };
}
const results = await response.json() as any;
```

For example, if the user prompts :

“Find me the IDs of all artifacts related to agriculture in the Uruk period.”

The LLM will execute it as :-

- Due to the mention of Uruk period, the LLM identifies, and uses the string “Uruk” in the “period” field, and in “atf\_translation\_text”, it will inject the value “agriculture”.
- LLM will generate a call to the advanced\_search handler like :

```
advanced_search({
  "period": "Uruk",
  "atf_translation_text":
  "agriculture",
  "limit": 50
})
```

- **Grounding** of the field values will occur (explained ahead)
- The server constructs a query URL to the CDLI API –

[https://cdli.earth/search.json?period=Uruk&atf\\_translation\\_text=agriculture&limit=50&offset=0](https://cdli.earth/search.json?period=Uruk&atf_translation_text=agriculture&limit=50&offset=0)

- Response is formatted as an array of entities, and the LLM can iterate through them for further exploration, if needed.

### 5.1.2 ground\_term

The user may make **typos** in the prompt they write, so the `advanced_search` tool internally perform **Tool Augmented Grounding**, and calls the unexposed tool **ground\_term**, that normalises the user provided term to the closest CDLI canonical value. It supports only the fields that are groundable, like provenience, period, collection, genre, language, etc.

It internally uses the concept of **Levenshtein Distance algorithm** to find the **minimum edit distance** between the provided term, and the terms from a pre-provided list to the `ground_term` tool.

```
export const name = "ground_term";
export const description = "Normalises a user-provided term to the closest CDLI canonical value.";
export const inputSchema = {
  type: "object",
  properties: {
    field: {
      type: "string",
      description: "One of: provenience, period, collection, genre, language"
    },
    value: {
      type: "string",
      description: "User-provided term (may contain typos)"
    }
  },
  required: ["field", "value"]
};

function levenshtein(a: string, b: string): number {
  // algorithm implementation to get the edit distance
}
```

It **works under the hood of `advanced_search`**, and is not exposed to the LLM, and works in the following sequence –

- The LLM extracts the term "Ur 3".
- `advanced_search` identifies this as a period.
- It calls the `ground_term` logic internally.
- `ground_term` realization: "Ur 3" approxes to "Ur III" (Distance = 2).
- The search is sent to the API using the correct string: `period=Ur%20III`.

### 5.1.3 get\_metadata

It **fetches the JSON metadata** for any entity, that is having its own id (eg. artifacts, publications, authors, dynasties, periods, etc).

It can be used in two distinct ways :-

- Looking up a **specific item using its id**
- Getting a **paginated list of all the items** within a category

Supported resources –

- **Core Entities:** *artifacts, publications*
- **Geographic/Temporal:** *proveniences, periods, dynasties*
- **People/Organizations:** *authors, rulers, archives, collections*
- **Physical/Categorical:** *materials, genres*

For example –

- to get details of a specific artifact :

```
"get_metadata"({  
  "resource": "artifacts",  
  "id": "254876"  
})
```

- to list the first 10 historical periods :

```
"get_metadata"({  
  "resource": "period",  
  "limit": "10"  
})
```

**Relevance:** While `advanced_search` is good for finding entities based on queries, `get_metadata` is the tool the LLM will use to further inspect things once they are found.

For example, if the user prompts :

“Retrieve the physical characteristics and museum information for artifact P12345.”

The URL generated for data fetching will be :

<https://cdli.earth/artifacts/12345>

```
export const inputSchema = {
  type: "object",
  properties: {
    resource: {
      type: "string",
      description: "The category to fetch. Supported paths include: artifacts, publications, authors, rulers, dynasties, periods, archives, collections, genres, materials, proveniences."
    },
    id: {
      type: "string",
      description: "The specific ID of the resource. If omitted, returns a paginated list of resources."
    },
    limit: {
      type: "number",
      description: "Number of resources to return if id is omitted (default: 20).",
    },
    page: {
      type: "number",
      description: "Page number for pagination (default: 1).",
    },
  },
  required: ["resource"]
};
```

### Tool Definition of get\_meradata

```
const baseUrl = process.env.CDLI_API_BASE_URL || "https://cdli.earth";
let url = `${baseUrl}/${args.resource}`;
if (args.id) {
  url += `/${args.id}`;
} else {
  const limit = args.limit ?? 20;
  const page = args.page ?? 1;
  url += `?limit=${limit}&page=${page}`;
}
const response = await fetch(url, {
  headers: {
    'Accept': 'application/json',
    'User-Agent': 'cdli-mcp-server/1.0.0'
  }
});
```

### API response fetching through get\_metadata tool

### 5.1.4 search\_entity

Some entities in the CDLI framework supports independent searching for their specific endpoints with some supported subqueries.

For example, [/authors](#) endpoint supports searching over queries like 'author', 'orcid' and 'institution', [/genres](#) support 'genre' and 'parent'.

This tool covers that use case, to let the LLM call this tool, when the user wants to get info about some entity with few filters applied (note that the filters need to be supported by the respective apis)

```
export const inputSchema = {
  type: "object",
  properties: {
    entity_type: {
      type: "string",
      enum: ["authors", "proveniences", "languages", "collections",
"materials", "regions", "periods", "genres"],
      description: "The type of entity to search for."
    },
    filters: {
      type: "object",
      additionalProperties: { type: "string" },
      description: "A key-value map of filters. Use 'name' for general
text search. E.g., {'name': 'alexander'}. Other specific keys: 'orcid'
(authors), 'region' (proveniences), 'parent' (languages)."
    },
    limit: {
      type: "number",
      description: "Number of results to return (default: 20)."
    },
    page: {
      type: "number",
      description: "Page number for pagination (default: 1)."
    }
  },
  required: ["entity_type", "filters"]
};
```

Tool definition of search\_entity

Here, few entities are shown, but it would be implemented to support all the available entities, and in future, if new entities are added in the framework, just appending it to the shown list would work.

The supported **subqueries are mapped to their respective entity** as shown below :

```
const FIELD_MAP: Record<string, Record<string, string>> = {
  authors: { name: "author", orcid: "orcid" },
  proveniences: { name: "provenience", region: "region" },
  languages: { name: "language", parent: "parent" },
  collections: { name: "collection" },
  materials: { name: "material" },
  regions: { name: "region" },
  periods: { name: "period" },
  genres: { name: "genre" }
};
```

### API response fetching :

```
const baseUrl = process.env.CDLI_API_BASE_URL || "https://cdli.earth";
const url = `${baseUrl}/${entity_type}?${params.toString()}`;
const response = await fetch(url, {
  headers: { 'Accept': 'application/json', 'User-Agent': 'cdli-mcp-server/1.0.0' }
});
if (!response.ok) {
  return { content: [{ type: "text", text: `API error searching ${entity_type}: HTTP ${response.status} - ${response.statusText}` }], isError: true };
}
```

Instead of the raw JSON, normalized JSON Is returned to the LLM, to maintain uniformity, and keeping the focus on the main fields for further exploration.

```

const entities: any[] = Array.isArray(data) ? data :
(Array.isArray(data.entities) ? data.entities : []);

// Normalize results
const normalizedResults = entities.map((entity: any) => {
  const id = entity.id || entity.author_id || entity.provenience_id ||
entity.language_id || entity.collection_id || entity.material_id ||
entity.region_id || entity.period_id || entity.genre_id ||
Object.values(entity)[0];
  const name = entity.name || entity.author || entity.provenience ||
entity.language || entity.collection || entity.material || entity.region ||
entity.period || entity.genre || entity.title || "Unknown";

  return {
    id: String(id),
    name: String(name),
    raw: entity
  };
});

const output = {
  results: normalizedResults,
  total: data.paging?.count || normalizedResults.length
};

```

### Normalization of API response

For example –

If the tool call looks like –

```

search_entity({
  "entity_type": "authors",
  "filters": {
    "name": "alexander"
  }
})

```

Result –

```

{
  "results": [
    {
      "id": "42",
      "name": "Alexander, Robert",
      "raw": { "author_id": 42, "author":
"Alexander, Robert", "orcid": "0000-..." }
    }
  ],
  "total": 1
}

```

(URL generated : <https://cdli.earth/authors?author=alexander>)



### 5.1.5 Other tools that will be implemented include

#### - `get_bibliography`

- **Description:** Pulls bibliography data for artifacts or publications in strictly academic formats (BibTeX or CSL-JSON).
- **Use-Case prompt:** "Generate a proper BibTeX citation for this artifact to append to the research paper."

#### - `get_inscription`

- **Description:** Fetches the actual physical text or transliteration of a specific inscription (supports C-ATF, CDLI-CoNLL, and CoNLL-U).
- **LLM Use Case:** "Read the transliteration of this tablet so I can summarize its contents or perform NLP grammar analysis."

#### - `cqp_query`

To run a linguistic query for a CDLI corpus.

```
export const name = "cqp_query";
export const description = "Run a CQP (Corpus Query Processor) linguistic pattern search against the CDLI corpus via the CQP-to-RDF API.";
export const inputSchema = {
  type: "object",
  properties: {
    expression: { type: "string", description: "A valid CQP expression, e.g. [lemma='lugal' & pos='N']" },
  },
  required: ["expression"]
};
```

Once the tools are built, the server needs to support two different ways to communicate, because it will be used from two very different environments:

- **STDIO transport:** Used by desktop AI clients (like Claude Desktop or Cursor) that run the server as a local background process on the researcher's own machine.
- **HTTP/SSE transport:** Used by the browser-based chat widget on the CDLI website, which needs a persistent connection to a publicly hosted server over the internet.

Connecting from Claude Desktop (or other local agents) requires only a single config entry:

```
"cdli": {
  "command": "npx",
  "args": ["-y", "@cdli/mcp-server"]
}
```

- The response that the server will get from the CDLI APIs can be very large, resulting in tens of thousands of token wastage, when most of the data in the response is irrelevant for a particular query.
- To tackle that, the response will be compressed and normalised before feeding it back to the LLM.

Before

```
{
  "artifact_id": 254876,
  "cdli_id": "P254876",
  "designation": "MVN 3, 1",
  "museum_no": "BM 12345",
  "accession_no": "1905-10-14, 1",
  "excavation_no": "ABC-001",
  "created_at": "2002-10-18T12:00:00Z",
  "updated_at": "2024-01-10T15:30:00Z",
  "contributor_id": 42,
  "is_published": true,
  "is_sealed": false,
  "proveniences": [
    {
      "provenience_id": 14,
      "provenience": "Nippur",
      "region": "Babylonia",
      "latitude": 32.12,
      "longitude": 45.14
    }
  ],
  "periods": [
    {
      "period_id": 6,
      "period": "Ur III",
      "full_name": "Third Dynasty of Ur"
    }
  ],
  "materials": [
    {
      "material_id": 1,
      "material": "Clay"
    }
  ],
  "genres": [
    {
      "genre_id": 3,
      "genre": "Administrative"
    }
  ],
  "external_links": {
    "digitized_at": "https://...",
    "wikidata_id": "Q12345"
  }
}
```

After

```
{
  "id": "P254876",
  "designation": "MVN 3, 1",
  "museum_no": "BM 12345",
  "provenience": "Nippur",
  "period": "Ur III",
  "material": "Clay",
  "genre": "Administrative"
}
```

## Testing :

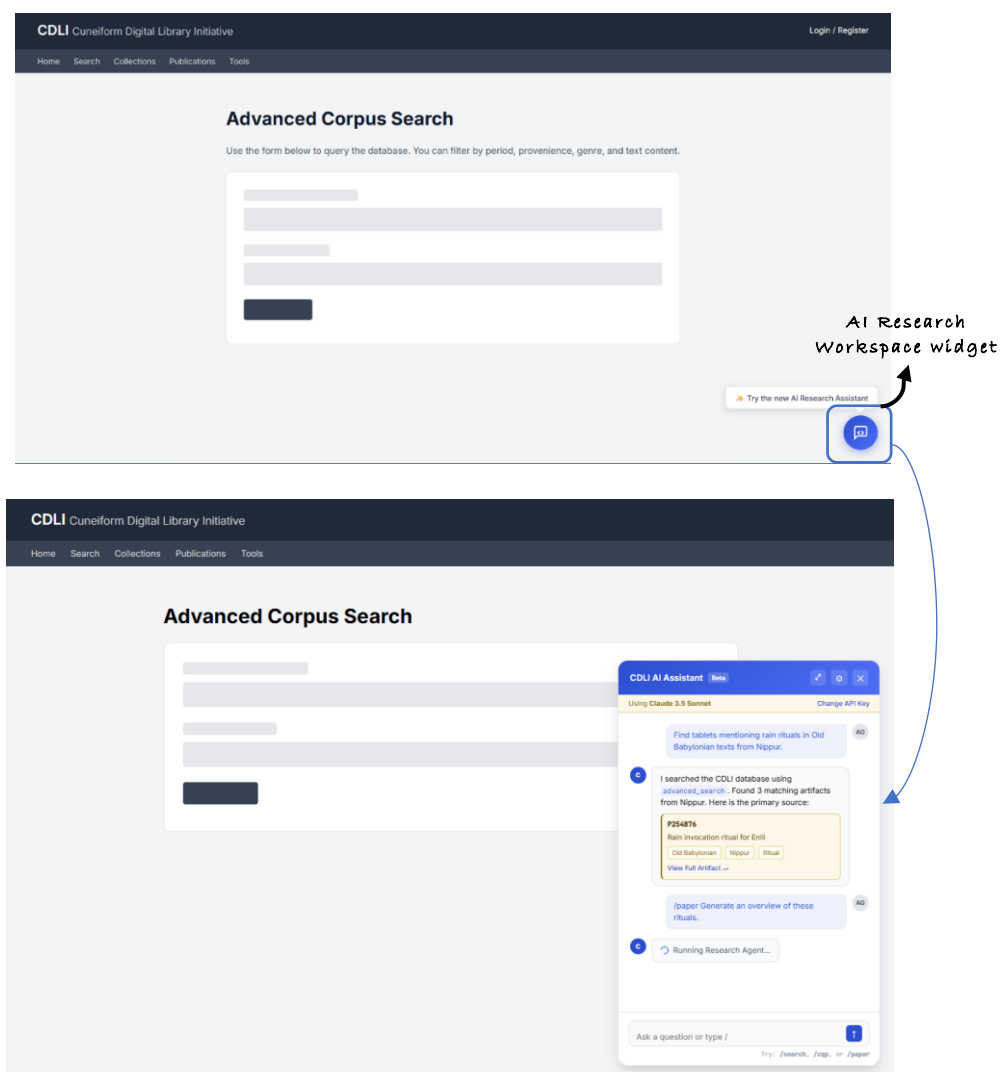
- Unit test for each tool and function will be written.
- Manual testing can be done with the officially available MCP Inspector.

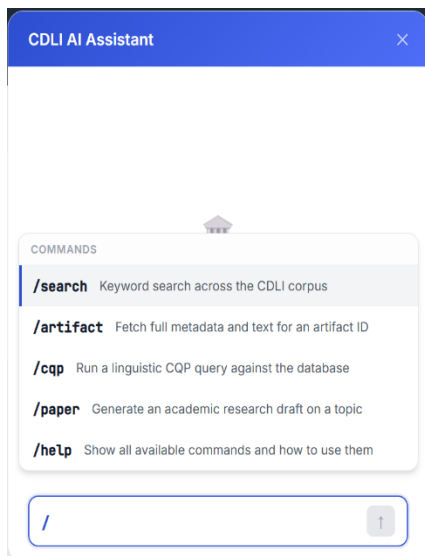
## 5.2 Developing the AI Research Workspace

The AI interface will be a **browser-based MCP client** that manages communication with the server over SSE, handles tool invocation responses, maintains session context, and renders structured outputs such as citations and artifact cards. It will be available on the CDLI website and opens a **chat drawer** on the side of the screen. It works on every page of the site with no separate navigation required.

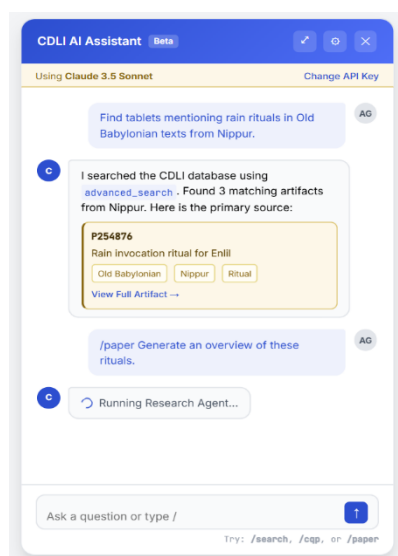
It will be the main MCP client available directly on the CDLI website.

### 5.2.1 Proposed Frontend

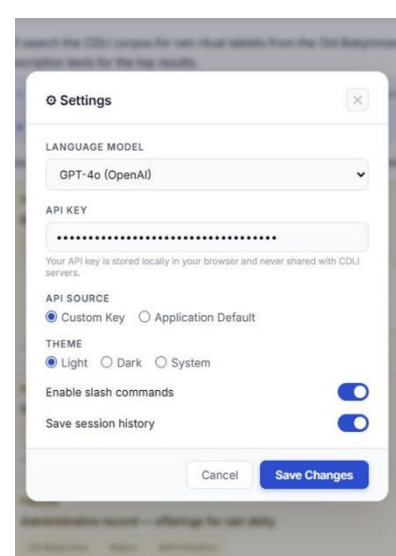




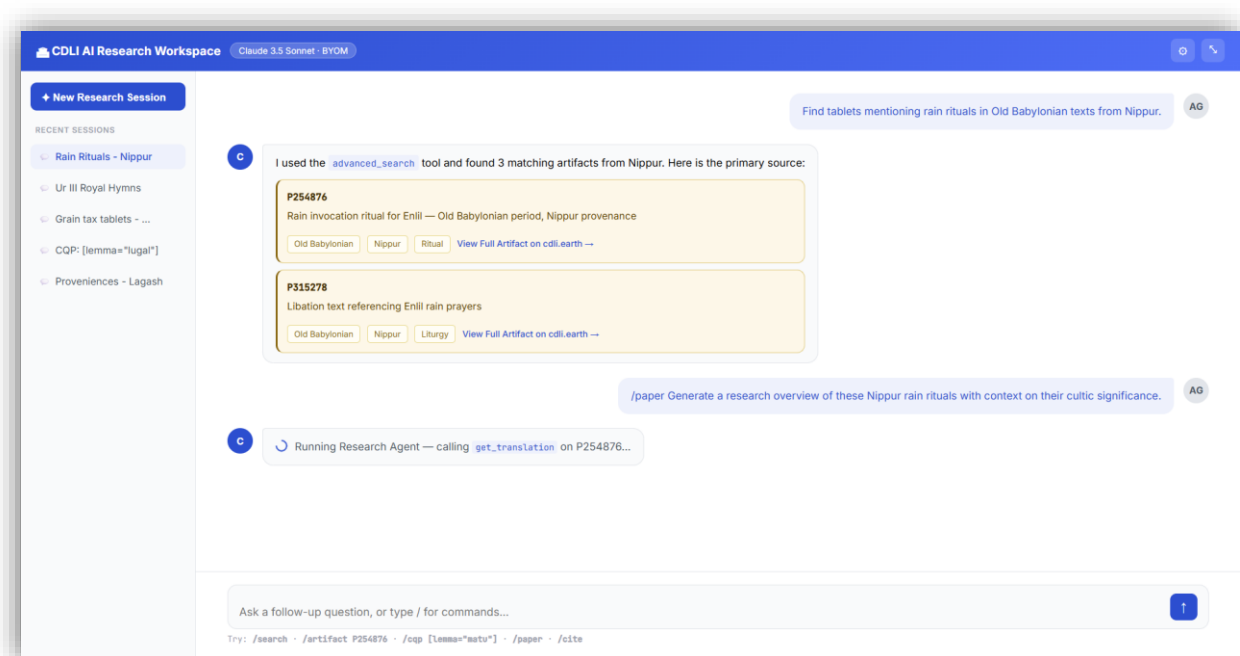
Slash Commands



Floating Widget



Settings Popup



AI Research Workspace Maximized

The widget connects to the hosted MCP server via SSE. Researchers enter their own API key (OpenAI, Anthropic, Gemini) in the settings panel. **The key stays entirely in the browser** and it never touches CDLI's servers. The widget supports:

- Markdown rendering for clean, readable responses
- Inline artifact **citation cards** (with artifact ID, period, and a direct link)
- **Slash commands** (`/search`, `/artifact`, `/cqp`, `/paper`) for quick structured workflows

### 5.2.2 API Key storage and encryption

To keep the **backend fully stateless** and avoid handling user credentials, API keys are managed entirely on the client using a local encrypted store.

- **Mechanism:** User API keys are **never sent to or stored on the server**. They are encrypted in the browser using **AES-256-GCM** through the **Web Crypto API**.
- **Key Derivation:** The encryption key is derived from a **user-defined session PIN** using **PBKDF2**.
- **Storage:** Only the encrypted key (ciphertext), along with the salt and initialization vector (IV), is stored in the browser's localStorage.
- **Security:** Since the server never sees the API keys, there is **no risk of server-side leakage**. The decryption key (PIN) is **not** stored and is only kept in memory during the session, which reduces exposure even if the page is compromised.

### 5.2.3 Context management during an ongoing session

The chat widget maintains the conversation state locally in the browser's memory. To ensure total cost predictability, the system enforces a strict FIFO sliding window.

- **History Structure:** Every new message from the user triggers a request that includes only the last 3 pairs of User: Query / Assistant: Response.
- **Context Benefit:** The history component of the context window is capped at a maximum of ~800 tokens, regardless of how long the user chats.

Along with it, since the token usage is going to be increased due to overhead being added with every conversation exchange, there would be a need to compress the Json responses. For that, **we will project the JSON into a compact version** containing only the most research relevant fields. (I have explained this approach above too, in the MCP server section.)

### 5.2.4 General Token Usage Breakdown

The widget is designed to be highly lightweight, ensuring that even large queries fit comfortably within the smallest modern context windows (e.g., **8k–16k tokens**) offered by free-tier or budget-friendly API models.

Below is the exact estimated token usage for a standard conversational turn, accounting for everything from tool schemas to API responses.

| Component              | Description                                                                                 | Approx. Tokens |
|------------------------|---------------------------------------------------------------------------------------------|----------------|
| Base System Prompt     | Core guardrails, persona instructions, and citation rules                                   | ~200           |
| MCP Tool Manifest      | Names, descriptions, and JSON Input Schemas for all 7 active CDLI tools                     | ~1,000         |
| Sliding Window History | The last 3 conversational turns (user queries + assistant text responses)                   | ~600           |
| Tool API Response      | The JSON payload returned from the CDLI database (e.g., <code>get_artifact</code> metadata) | ~400           |
| Current User Prompt    | The immediate leading prompt or question from the user                                      | ~50            |
| Total (Standard Turn)  | Average context payload sent to the LLM per conversational turn                             | ~2,250         |

The token usage may peak at 4k-5k tokens per convo-turn. This ensures under-budget, reliable and stable API execution for users bringing their own API keys.

## 5.3 The Research Paper Generation Agent

The `/paper` command is the most sophisticated slash command. It triggers a **multi-phase agentic workflow** that constrains the LLM to a safe, context-aware pipeline, and thus preventing the context-window explosion that would occur if the agent tried to ingest dozens of full artifact transliterations at once.

When a user types `/paper <topic>`, the widget intercepts the input and injects a workflow that is orchestrated programmatically using a **LangGraph** state machine on the client side (e.g., the middleware serving the chat widget).

The **LangGraph Orchestrator** manages the multi-turn workflow, controls which tools the LLM can use at each step, isolates context windows to prevent hallucinations, and streams progress updates back to the UI.

### 5.3.1 Graph state definition

For keeping the memory for the entire workflow, the LangGraph passes a **PaperState** object between different nodes, so that the nodes receive the relevant data.

```

class ArtifactBrief(TypedDict):
    id: str          # P-number e.g. "P010451"
    title: str        # Designation or Museum No
    period: str       # e.g. "Ur III"
    provenience: str # e.g. "Girsu"

class Theme(TypedDict):
    name: str
    supporting_artifacts: list[str]
    summary: str

class PaperState(TypedDict):
    topic: str          # User's query
    found_artifacts: list[ArtifactBrief] # Raw Discovery results
    shortlisted_artifacts: list[ArtifactBrief] # Top 10 ranked
    artifact_summaries: list[str]          # Concise interpretive summaries
    themes: list[Theme]                   # Clustered themes
    draft: str                          # Final Markdown synthesis
    errors: list[str]                    # Collected issues

```

Graph state definition

### 5.3.2 Agentic Nodes

The workflow is broken down into **four distinct phases**, configured as LangGraph nodes. Each node equips the LLM with a specific system prompt and an isolated subset of tools.

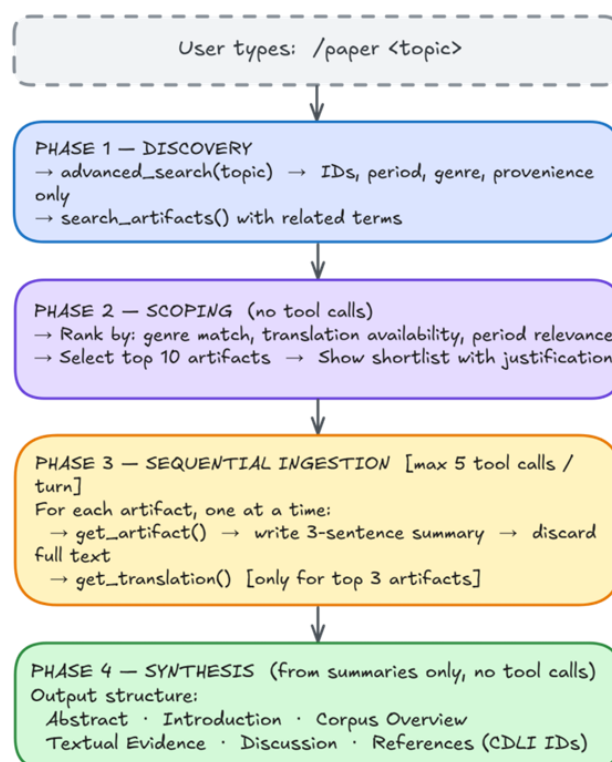


Fig. Phases of paper generation workflow

## Node 1: Discovery

- **Input:** topic
- **Tools:** advanced\_search
- **Role:** Generates 1–3 broad search queries from the topic to explore the CDLI corpus. Collects artifact IDs and basic metadata.
- **Output:** Stores results in found\_artifacts

## Node 2: Scoping & Ranking

- **Input:** topic, found\_artifacts
- **Tools:** None
- **Role:** Filters and ranks artifacts using metadata such as genre, period relevance, and availability of translations. Limits the set to the top 10.
- **Output:** Stores results in shortlisted\_artifacts

## Node 3: Ingestion & Summarization

- **Input:** shortlisted\_artifacts, topic
- **Tools:** get\_metadata, get\_inscription
- **Role:** Iterates over each artifact:
  - fetches metadata and transliteration
  - generates a short ( $\approx 3$  sentence) summary focused on the topic
  - discards raw transliteration after use to save context
- Handles timeouts without stopping the workflow
- **Output:** Stores summaries in artifact\_summaries, logs issues in errors

## Node 3.5: Thematic Clustering

- **Input:** artifact\_summaries

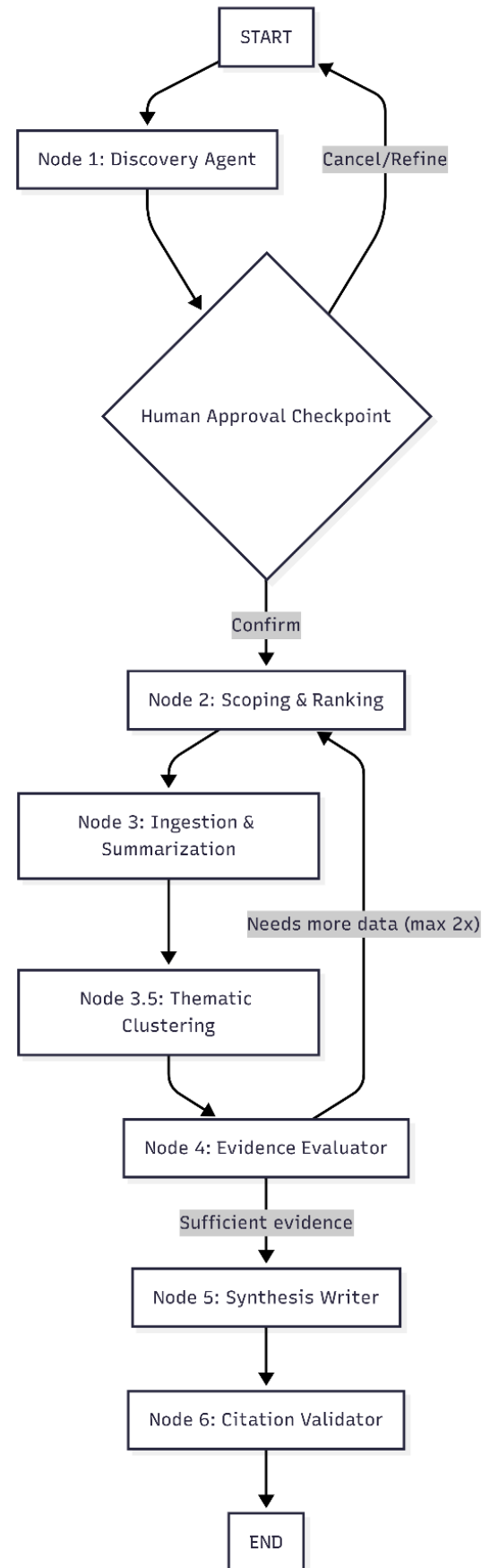


Fig. Agentic Workflow of /paper



- **Tools:** None
- **Role:** Identifies patterns across summaries and groups them into themes. Each theme includes a label, a short explanation, and supporting artifact IDs.
- **Output:** Structured themes array

#### Node 4: Evidence Evaluation

- **Input:** themes, evaluation\_attempts
- **Tools:** None
- **Role:** Checks if the themes are meaningful and diverse enough. **If coverage is weak** (e.g., too few artifacts or vague grouping), **it triggers another iteration** starting from scoping. Limited to 2 retries.
- **Output:** needs\_more\_research flag

#### Node 5: Synthesis Writer

- **Input:** topic, themes
- **Tools:** None
- **Role:** Writes the final research output using only the structured themes. Raw data is not accessible at this stage to avoid overload. Each paragraph must include at least one valid CDLI citation.
- **Output:** Generates draft (Markdown)

#### Node 6: Citation Validator

- **Input:** draft, shortlisted\_artifacts
- **Tools:** None
- **Role:** Verifies all cited artifact IDs against the known set. Any unknown or invalid references are flagged and added as warnings.
- **Output:** Final draft + citation\_issues

Since LangGraph provides updates at each step of workflow, **the chat widget can show progress** to the user, instead of a boring loader.

## 5.4 Deployment, Testing & Documentation

### 5.4.1 Anticipated Technical Challenges & Mitigations

| CHALLENGE                               | MITIGATION                                                                                                                                 |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CDLI API RATE LIMITS</b>             | Cache frequent identical queries in memory for ~30 seconds                                                                                 |
| <b>LLM GETTING STUCK IN A TOOL LOOP</b> | Hard limit of 5 consecutive tool calls per user turn; a structured error is returned after that so the LLM can gracefully inform the user. |
| <b>SLOW API RESPONSES</b>               | Every CDLI fetch is wrapped in an <a href="#">AbortController</a> with an 8-second timeout; the LLM is notified if a call times out.       |
| <b>SSE CONNECTION DROPS</b>             | The client widget auto-reconnects with exponential backoff.                                                                                |
| <b>OVERLY LONG TOOL RESPONSES</b>       | Tool output is trimmed before being fed back to the LLM. Pagination is used for large result sets.                                         |

### 5.4.2 TESTING AND VALIDATION

Testing happens at three levels:

**a) Unit Tests (per tool)** - Every MCP tool handler will be tested using mocked CDLI API responses, checking that inputs are validated, outputs are formatted correctly, and edge cases (empty results, invalid IDs) are handled cleanly.

**b) Integration Tests** - The official MCP Inspector tool will be used to test the live SSE endpoint

**c) E-2-E Tests** - A set of pre-defined researcher queries (e.g., *"Find tablets from Nippur in Sumerian"*) will be run against the live server to validate:

- Search results returned in **under 2 seconds**.
- Translation fetch returned in **under 3 seconds**.

Latency is measured using timing wrappers around every tool call that log response time to the console, making it easy to spot regressions during development.

### 5.4.3 DEPLOYMENT

- Packaging the server with a **DockerFile**.
- Testing on Cloud Platforms (e.g. AWS) –  
We can use **AWS EC2** and **AWS Lambda** for backend testing, with the environment being similar to production.
- After merging with the production, the server can be hosted on the **CDLI cloud infra**.

## 6 PROJECT TIMELINE

| Duration            | Phase                                | Description                                                                                                                                                                                                                |
|---------------------|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Week 1 – 2          | Core MCP Server & Tool Design        | Study the CDLI APIs and framework. Finalizing the MCP server design (tools, req/res format). Setting up the basic server using MCP SDK. Defining a standard format for all tools.                                          |
| Week 3 – 4          |                                      | Implement the advanced_search tool. Add grounding logic to handle typos. Finishing the implementation of tools like search_entity & get_metadata. Testing the tool behaviour using mcp inspector                           |
| Week 5              |                                      | Design and implement response compression layer to extract the relevant fields and normalizing the output across tools. Optimize the responses to reduce token usage.                                                      |
| Week 6              |                                      | Implement STDIO and HTTP/SSE transport modes. Upto now, the core MCP server is mostly complete. Testing the server in Inspector, and in Claude Desktop using STDIO. Dockerize the server and test remote access using SSE. |
| MID-TERM EVALUATION |                                      |                                                                                                                                                                                                                            |
| Week 7 – 8          | Developing the AI Research Workspace | Moving forward to implement the client, the chat widget. Building the interface using React. Connect it to MCP server via SSE. Implement API key handling, and context management for running session.                     |

|                            |                                                |                                                                                                                                                                                                                                    |
|----------------------------|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Week 9 – 10                | <b>Implementing the Paper Generation Agent</b> | Design and implement the <b>/paper Research Paper Generation Workflow</b> using <b>LangGraph</b> . Add structured steps for discovery, filtering, summarizing and drafting. Ensure controlled tool usage & citation handling.      |
| Week 11                    | <b>Deployment, Testing &amp; Documentation</b> | <b>Integrate the widget into the CDLI website</b> locally. Write unit tests for end-to-end testing across server, tools, and client. Handle edge cases (as there will be a lot of them), and try to <b>optimize performance</b> .  |
| Week 12                    |                                                | Preparing for soft-deployment, on CDLI-like cloud infra. <b>Complete the documentation</b> and usage guides. Making the server available to be installed. Incorporate <b>feedback from the community and finalize submission</b> . |
| <b>END-TERM EVALUATION</b> |                                                |                                                                                                                                                                                                                                    |

## 7 PLANNED GSoC WORK HOURS

To achieve the above outcomes, it will take **175 hrs**.

## 8 MORE ABOUT ME

I am Armaan Gupta, an Electronics and Communication Engineering major at **IIITDM Jabalpur**, India (Class of **2027**). I have over 1.5 years of experience building Full Stack Applications and am well-versed in **Java, TypeScript, C++, Golang, and Python**. My strong academic background, including **excellent grades in Probabilistic Approaches to ML, Data Structures and Algorithms and AI and its Applications** makes me a strong candidate for contributing to this project.

For this project, I have already developed a **working proof-of-concept** MCP server for CDLI. This prototype allowed me to explore the MCP SDK's tool registration model, CDLI API response formats, and stdio/HTTP transport mechanisms. This hands-on experience demonstrates my readiness to build the full production implementation with a focus on robust architecture, testing, and documentation.

I have been contributing to cdli/framework for over a month now. During this duration, I have been working on multiple issues, listed below –

| Sr No. | Issue Link            | Merge Request Link    | Small Description                                            | Status |
|--------|-----------------------|-----------------------|--------------------------------------------------------------|--------|
| 1.     | <a href="#">#2458</a> | <a href="#">!1119</a> | Updated MySQL commands to MariaDB ones.                      | Merged |
| 2.     | <a href="#">#2448</a> | <a href="#">!1121</a> | Refined new user welcome emails                              | Merged |
| 3.     | <a href="#">#2531</a> | <a href="#">!1150</a> | Updated export dialog linking to CDLI API Client             | Merged |
| 4.     | <a href="#">#2495</a> | <a href="#">!1120</a> | Added filters to hide empty author profiles from public view | Open   |

As far as engaging with the open-source community is concerned, I am also moderating the open communication forum (Slack) of CDLI.

Please find my resume here : [resume](#)

## 9 COMMITMENTS AND AVAILABILITY

I will be on a summer break from May to July, during this time and I will be able to commit ~40 hours/week. From 31st July onward, I will only be able to commit ~25 hours/week. I may increase the hours based on the pace/requirements of the project.

## 10 AI BEING THE HELPING HAND

I use tools like **Antigravity** and **Claude** regularly while working, mainly to speed things up and think through problems, not to replace actual implementation.

When I start something new, I usually use them to **brainstorm** different approaches. For example, while designing the MCP tools and the /paper workflow, I used them to explore different ways to structure things before deciding what makes sense.

## 11 REFERENCES

- MCP TypeScript SDK:  
<https://github.com/modelcontextprotocol/typescript-sdk>
- MCP Build a Server Guide:  
<https://modelcontextprotocol.io/docs/develop/build-server>
- Introduction to MCP (Anthropic):  
<https://anthropic.skilljar.com/introduction-to-model-context-protocol>
- mcp-open-library (reference architecture):  
<https://github.com/8enSmith/mcp-open-library>

\*\*All supplementary materials, including **UI mock-up images, sequence diagrams**, and the proposed directory structure with sample files, are available at: <https://github.com/armaanngupta/cdli-mcp-server-assets>

\*\*A **working prototype** with fewer tools is available [here](#).

Thank you for going through my proposal :)